

Naïve Bayes Software Defect Prediction Model

WANG Tao, LI Wei-hua
School of Computer Science and Technology,
Northwestern Polytechnical University,
Xi'an, China
water@snnu.edu.cn

Abstract— Although the value of using static code attributes to learn defect predictor has been widely debated, there is no doubt that software defect predictions can effectively improve software quality and testing efficiency. Many data mining methods have already been introduced into defect predictions. We noted there have several versions of defect predictor based on Naïve Bayes theory, and analyzed their difference estimation method and algorithm complexity. We found the best one which is Multi-variants Gauss Naïve Bayes (MvGNB) by performing prediction performance evaluation, and we compared this model with decision tree learner J48. Experiment results on the benchmarking data sets of MDP made us believe that MvGNB would be useful for defect predictions.

Keywords— *software defect prediction; software metrics; Naïve Bayes; statistical methods;*

I. INTRODUCTION

Software defect is an error situation of software system. This kind of error situation is caused by explicit or potential violation of security policies at runtime because of wrong specifying and inappropriate development or configuration [1]. In practice, defects of software system include design flaw, execution vulnerability and operation error. Software defects severely threaten running security of software system, so defect detection and prediction directly dominate the development cost and running security of its. Defect detection strives to find defects, and defect prediction strives to predict unknown defects instead. Defect prediction can help engineers find some important information earlier about location, quantity and distribution of defects, and so engineers can improve test efficiency and

software quality. Using software metrics or code attributes to predict quantity and distribution of defects is major method of defect prediction which needs to extract software metrics, compute attribute values and construct prediction model based on software metrics. In order to construct prediction model, many data mining methods were used by researchers such as regression analysis, neural network and Bayesian theory etc. Although some researchers underestimated and even questioned the effect and significance of software prediction, such as Zhang [2] oppugned Tim Menzies's study results in [3] and considered his theory is useless in practice, many studies [4,5,6,7,8,9,10] have proved significance of defect prediction. Fenton [4] proposed a cause-effect relationship between unknown defects and known defects and test validity, and he used Bayesian networks to construct the prediction model. Tibor Gyimóthy[5] applied four methods including logistic regression, linear regression, decision tree and neural network to do defect prediction on an open source project data set, he found that using Coupling Between Object classes(CBO) have best prediction performance. Song [6] proposed association rule mining based methods to predict defect associations and defect correction effort, and he emphatic analyzed the cost of defect correction.

Naïve Bayes(NB) is a very effective machine learning method. A NB model treats defect prediction as binary classification, it trains and constructs predictor by analyzing historical data of software modules, based on predictor it will make decision whether new module has defects or not. This paper analyzed some key points in Naïve Bayes software defect prediction model. To motivate others to compare experiment results, we did our experiments all on benchmarking data sets MDP [11]. We compared Naïve Bayes software defect prediction model with decision tree J48 which was also used in [3, 5]. Experiment results showed that the performance of Naïve Bayes is obviously better and proved effectiveness and significance of Naïve Bayes for software defect prediction.

II. NAIVE BAYES MODEL

A. Overview

Naïve Bayes Prediction (NBP) model chooses software module to be object unit of training and prediction. According to definition of IEEE [12], software module is 1) A program unit that is discrete and identifiable with respect to compiling, combining with other units, and loading; 2) A

This research is supported by the National High-Tech Research and Development Plan of China under grant No.2006AA01Z406, also supported by the Doctorate Foundation of Northwestern Polytechnical University under grant No.CX200815, the Planned Science and Technology Project of ShaanXi Province, China under grant No. 2010JM8039 and the Youth Foundation of ShaanXi Normal University under grant No.200701016.

logically separable part of a program. Let $A = \{a_1, a_2, \dots, a_n\}$ be set of metrics attribute set, there is a vector $M: \{(a_1, w_1), (a_2, w_2), \dots, (a_n, w_n)\}$ to denote a software module, where a_i is metrics and w_i is weight of a_i . We train the classifier using module data sets with category tag, and compute the defective probability of a new module which will make an alert when it exceeds a threshold.

If we define the category notion of software module is $c \in \{C_d, C_n\}$, where C_d is defective category and C_n is non-defective category. Then according to Bayesian theory [13], the probability that software module is defective will be computed by (1):

$$P[C_d | M] = \frac{P[M | C_d] \times P[C_d]}{\sum_{c \in C} P[c] \times P[M | c]}. \quad (1)$$

As shown by (2), classifier will classify software module M to C_d when the ratio greater than λ ,

$$\frac{P[C_d | M]}{P[C_n | M]} > \lambda. \quad (2)$$

$P[M | c]$ and $P[c]$ in (1) will be estimated using training data sets. Where $P[c]$ will be computed by (3):

$$P[c] = \frac{N_c}{N}, \quad (3)$$

where N_c is training modules number of category c and N is total number of training modules. Estimation of $P[M | c]$ is a key problem of NBP model. Some kinds of NBP models are derived from deferent estimation method of $P[M | c]$ by researchers.

B. Multinomial Naïve Bayes

Multinomial Naïve Bayes(MNB) is the most common estimation model of $P[M | c]$. In Vector Space Model w_i is attribute value of metric a_i in M . In effect, this is a unigram language model [13]. Furthermore, each software module $M = \{(a_1, w_1), (a_2, w_2), \dots, (a_n, w_n)\}$ of category c is seen as the result of picking independently n attributes from A with replacement, with probability $P[a_i | c]$ for each a_i . Then, $P[M | c]$ is the multinomial distribution:

$$P[M | c] = P(|M| \times |M|!) \times \prod_{i=1}^n \frac{P[a_i | c]^{w_i}}{w_i!}. \quad (4)$$

From (1), we found that $P(|M| \times |M|!)$ is divided in (4), so we did not care the representation and computation of $|M|$. Then from (1) and (4) we have:

$$P[C_d | M] = \frac{P[C_d] \cdot \prod_{i=1}^n P[a_i | C_d]^{w_i}}{\sum_{c \in C} P[c] \cdot \prod_{i=1}^n P[a_i | c]^{w_i}}. \quad (5)$$

Equation (5) is computation formula of MNB. Each $P[a_i | c]$ is estimated by (6) which is smoothed by Laplacean[13],

$$P[a_i | c] = \frac{1 + N_{a,c}}{n + N_c}, \quad (6)$$

where $N_{a,c}$ is the sum of w_i of the i -th metric a_i in training data sets category c , and n is number of metrics N_c is sum of w_i of all metrics in training data sets category c .

MNB have followed the “naïve” assumption that metrics does not depend on the category c . At the same time we supposed scale of software modules $|M|$ does not depend on the category c . this is an additional over-simplistic assumption, which is more questionable in classification, because large scale software modules common cause large defective probability.

C. Multi-variants Gauss Naïve Bayes

When attribute value of metrics is real-valued, Multi-variants Gauss Naïve Bayes (MvGNB) estimates $P[M | c]$ using (7):

$$P[M | c] = \prod_{i=1}^n g(w_i; \mu_{i,c}, \sigma_{i,c}), \quad (7)$$

where we supposed that each attribute follows a normal distribution $g(w_i; \mu_{i,c}, \sigma_{i,c})$ in each category c , where:

$$g(w_i; \mu_{i,c}, \sigma_{i,c}) = \frac{1}{\sqrt{2\pi}\sigma_{i,c}} e^{-\frac{(w_i - \mu_{i,c})^2}{2\sigma_{i,c}^2}}, \quad (8)$$

and the mean ($\mu_{i,c}$) and typical deviation ($\sigma_{i,c}$) of each distribution are estimated from the training data sets.

D. Flexible Bayes

Multi-variants Gauss Naïve Bayes has an assumption that each attribute follows a normal distribution, but this may not be satisfied in practice. Flexible Bayes (FB) models $P[M | c]$ as the average of $L_{i,c}$ normal distributions with different mean values, but the same typical deviation:

$$P[M | c] = \frac{1}{L_{i,c}} \cdot \sum_{l=1}^{L_{i,c}} g(w_i; \mu_{i,c,l}, \sigma_{i,c}), \quad (9)$$

where $L_{i,c}$ is the number of different values w_i has in the training data sets of category c . Each of these values is used as the mean $\mu_{i,c,l}$ of a normal distribution of that category. All the distributions of a category c are taken to have the same typical deviation, estimated as $\sigma_c = 1/\sqrt{N_c}$, where N_c is the number of training modules in c . If the assumption that the attributes follow normal distributions is violated in practice, FB can approximate the true distributions of real-valued attributes more closely than the MvGNB by averaging $L_{i,c}$ normal distributions.

III. EXPERIMENTAL DESIGN

A. Data sets

This paper did experiments and draw conclusions all based on NASA IV&V Facility Metrics Data Program (MDP) data sets. Many researchers employed these data sets [3, 7, 14, 15] which were collected from many real software projects of NASA. MDP have already become benchmarking data sets so that we can conveniently compare results between various studies by using different models and algorithms. As Table I shown, MDP include several projects coding with different languages. This paper selected CM1, JM1, KC1, and PC1 from MDP to do our evaluations.

TABLE I. DATA SETS IN MDP

Data set	Language	LOC	Modules	Defective
CM1	C	20k	505	45
JM1	C	315k	10878	899
KC1	C++	43k	2107	109
KC3	Java	18k	458	13
KC4	Perl	25k	125	75
MC1	C/C++	63k	9466	5
MW1	C	8k	403	24
PC1	C	40k	1107	90
PC2	C	26k	5589	22
PC3	C	40k	1563	167
PC4	C	36k	1457	126

TABLE II. INDICATORS OF ACCURACY

Name	Calculation
Recall, R	$TP/(TP+FN)$
Precision, P	$TP/(TP+FP)$
False Alarm, FA	$FN/(FP+TN)$
F-Measure	$2 * R * P / (R + P)$

B. Indicators of accuracy

Indicator of accuracy is also important in benchmarking experiment, and definition of indicators must be appropriate in order to it can true characterize the performance of prediction models. Based on common assumption of binary classification in data mining, the number of true positives (TP) is the number of defective modules that have been classified as defective, and similarly for false positives (FP, non-defective misclassified as defective), true negatives (TN, correctly classified non-defective), and false negatives (FN, defective misclassified as non-defective). So in our evaluations, we used some indicators shown as Table II.

We also used some another two indicators include ROC and AUC [16, 17]. The ROC graph is a 2D illustration of *Recall* rate on the Y-axis versus *False Alarm* rate on the X-axis [18]. The advantages of the ROC analysis are its

robustness toward imbalanced class distributions and to varying and asymmetric misclassification costs [7]. An ROC curve is obtained by varying the classification threshold over all possible values. So when two studies that use the same classifiers and data sets come to different conclusions just because of different procedures for determining classification thresholds are employed, ROC analysis can easily avoid this risk. AUC is area under the ROC curve. Higher AUC values indicate that the classifier is on average more to the upper left region of the graph.

C. Methodology and Results

1) Which Naïve Bayes?

This paper has already discussed the computing method of varying NBP models. In order to evaluate performance of these models, we compared prediction results classify on data sets used in this study. As Fig.1-Fig.4 shown, every model has similar performance on each data set. But on same data set performance of varying models has obvious deference. Furthermore, MNB shows worst prediction performance. FB and MvGNB are very close especially in large data set, i.e. JM1. Nevertheless, we noted that FB's algorithm complexity is $O(n \cdot |M|)$ same as MvGNB on training time, but on predicting time FB has $O(n \cdot |M|)$ algorithm complexity at the same time MvGNB's algorithm complexity is only $O(n)$, and where $|M|$ is number of modules in the data set. Therefore, we got the best performance NBP models within the MDP data sets context, which is Multi-variants Gauss Naïve Bayes.

2) Naïve Bayes vs. J48

Another evaluation was carried on in our studies to verify predictive ability of MvGNB within defect prediction context. We did 10-fold cross validation for MvGNB and compared performance with a state-of-art decision tree learner (J48 [19]) on 4 data sets above mentioned. We found MvGNB has shown better performance than J48 on each data set, especially on CM1 and KC1 AUC value of MvGNB is almost 10% higher than J48. Detail data are shown in Table III.

IV. CONCLUSION

Software defect prediction has become more and more important in software engineering especially in software system security. This paper introduced classic machine learning method Naïve Bayes in defect prediction, and analyzed construction of prediction models. Experiment results on benchmarking data sets show Multi-variants Gauss Naïve Bayes has best performance in all kinds of Naïve Bayes, and comparative experiment with J48 proved effectiveness of MvGNB in defect prediction. Next, we will introduce other machine learning methods into defect prediction and find best prediction model based on benchmarking experiments.

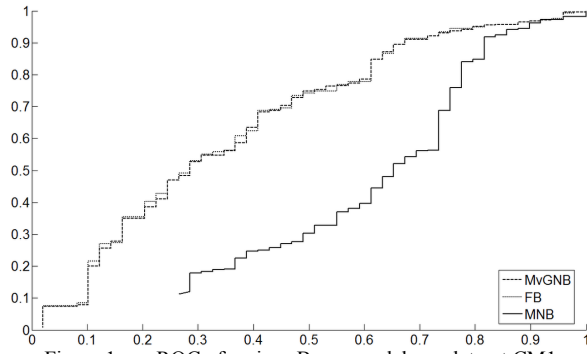


Figure 1. ROC of various Bayes models on dataset CM1

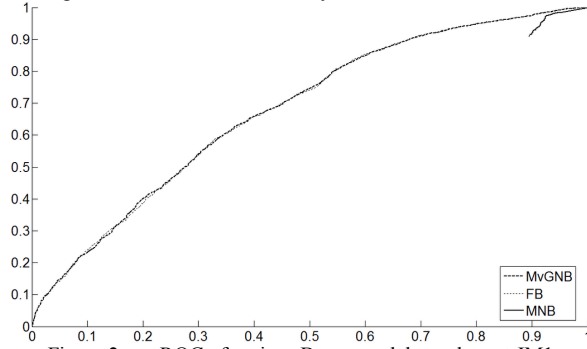


Figure 2. ROC of various Bayes models on dataset JM1

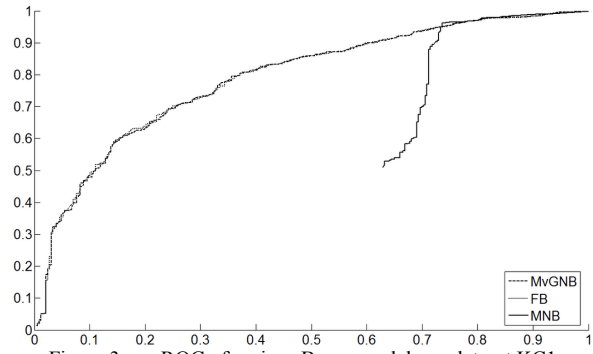


Figure 3. ROC of various Bayes models on dataset KC1

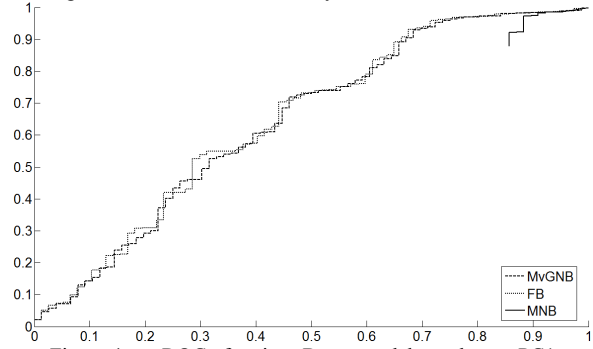


Figure 4. ROC of various Bayes models on dataset PC1

TABLE III. Performance of prediction model on various datasets

	Model	Recall	False Alarm	Precision	F-Measure	AUC
CM1	MvGNB	0.969	0.673	0.925	0.946	0.656
	J48	0.911	0.939	0.904	0.907	0.558
JM1	MvGNB	0.948	0.798	0.832	0.886	0.679
	J48	0.93	0.768	0.835	0.879	0.653
KC1	MvGNB	0.939	0.623	0.888	0.913	0.789
	J48	0.905	0.669	0.885	0.895	0.689
PC1	MvGNB	0.985	0.766	0.945	0.965	0.668
	J48	0.935	0.711	0.947	0.941	0.643

REFERENCES

- [1] M. Dowd, J. McDonald, and J. Schuh, "The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities," Boston, MA: Addison-Wesley, 2007.
- [2] Hongyu Zhang and Xiuzhen Zhang, "Comments on 'Data Mining Static Code Attributes to Learn Defect Predictors'," IEEE Transactions on Software Engineering, vol.33, no.9, pp.635-636, Sep., 2007.
- [3] T. Menzies, J. Greenwald and A. Frank, "Data Mining Static Code Attributes to Learn Defect Predictors," IEEE Transactions on Software Engineering, vol.33, no.1, pp.2-13, Jan., 2007.
- [4] Fenton NE, Krause P and Neil M, "Software Measurement: Uncertainty and Causal Modeling," IEEE Software, vol.10, no.4, pp. 116-122, Aug., 2002.
- [5] S. T. Gyimothy, R. Ferenc, and L. Siket, "Empirical Validation of Object-Oriented Metrics on Open Source Software for Fault Prediction," IEEE Transactions on Software Engineering, vol.31, no. 10, pp. 897-910, Oct., 2005.
- [6] Q. Song, M. Shepherd, M. Cartwright and C. Mair, "Song Software defect association mining and defect correction effort prediction," IEEE Transactions on Software Engineering, vol. 32, no. 2, pp. 69-82, Feb., 2006.
- [7] S. Lessmann, B. Baesens, C. Mues and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," IEEE Transactions on Software Engineering, vol.34, no.4, pp.485-496, Aug., 2008.
- [8] A. Güneş Kuru and Hongfang Liu, "Building Effective Defect-Prediction Models in Practice," IEEE Software, vol. 22, no. 6, pp.23-29, November/December, 2005.
- [9] I. Myrtveit, E. Stensrud, and M. Shepherd, "Reliability and Validity in Comparative Studies of Software Prediction Models", IEEE Transactions on Software Engineering, vol. 31, no. 5, pp 380-391, May, 2005.
- [10] Yue Jiang, Bojan Cukic, Tim Menzies and Nick Bartlow, "Comparing Design and Code Metrics for Software Quality Prediction," Proceedings of PROMISE'08, May 12-13, 2008, Leipzig, Germany.
- [11] M. Chapman, P. Callis and W. Jackson, "Metrics Data Program," NASA IV and V Facility, <http://mdp.ivv.nasa.gov/>, 2004.
- [12] IEEE, "IEEE Standard Glossary of Software Engineering Terminology," IEEE Standards Board, September, 1990:49.
- [13] Ian H. Witten and Eibe Frank, "Data Mining Practical Machine Learning Tools and Techniques Second Edit," San Francisco, CA: Morgan Kaufmann Publisher, 2005:88-97.
- [14] T. M. Khoshgoftar, N. Seliya and N. Sundares, "An Empirical Study of Predicting Software Faults with Case-Based Reasoning," Software Quality J., vol.14, no.2, pp.85-111, Feb., 2006.
- [15] S. Zhong, T. M. Khoshgoftar and N. Seliya, "Analyzing Software Measurement Data with Clustering Techniques," IEEE Intelligent Systems, vol.19, no.2, pp.20-27, Mar./Apr.2004.
- [16] Bradleya, "The use of the area under the ROC curve in the evaluation of machine learning algorithms," Pattern Recognition, vol.30, no.7, pp.1145-1159, July, 1997.
- [17] F. Provost and T. Fawcett, "Robust Classification for Imprecise Environments," Machine Learning, vol.42, no.3, pp.203-231, 2001.
- [18] T. Fawcett, "An Introduction to ROC Analysis," Pattern Recognition Letters, vol.27, no.8, pp.861-874, 2006.
- [19] J. R. Quinlan, "C4.5: Programs for Machine Learning," Morgan Kaufmann, 1993.